A package for PLATE RECognition In PYthon

Pejvak Javaheri

Version 0.1.0, documentation updated on January 8, 2025

Contents

1 Introduction

intro (Grady, 2006) Grady (2006) (Grady, 2006)

2 Dependencies

Currently, Linux is the only supported platform. Since MacOS is Unix-based, `platerecipy` is expected to be installed and used without a problem. The main reason that Windows is not supported at this point is that there some functionalities are implemented in `C` and require a slightly different compilation and linking procedure. Until the first release, availability of `platerecipy` on Windows is not a development priority.

Concerning the `Python` dependencies, `pip` should automatically install the necessary packages. Nonetheless, the following includes the necessary dependencies:

- `numpy` (Harris et al., 2020): for array operations
- `scipy` (Virtanen et al., 2020): for specific scientific utilities
- `matplotlib` (Hunter, 2007): for basic plots
- `pyvista` (Sullivan & Kaszynski, 2019): for `vtk` output and 3D visualization
- `scikit-image` (van der Walt et al., 2014): for a Random Walker implementation

In future releases, the Random Walker from `scikit-image` will be replaced by a `C` implementation tailored for problems that `platerecipy` is designed for. Until then, the `scikit-image` remains as a dependency.

3 Installation

Until the packages is released on PyPI, `platerecipy` can be installed using `pip` on the shell as follows:

```
cd dist/
python -m pip install platerecipy-?.?.?.tar.gz
```

where `platerecipy-?.?.?.tar.gz` is the installation tar file for the desired version. Note that all installation tar files can be found in `dist` subdirectory.

4 Basic interface

`platerecipy` is developed under the notion that it must accommodate users with different dataset formats and from different grid structures (e.g., spherical, YinYang, icosahedral, etc.). Accordingly, as much as possible, the interface must refrain from assuming any format or structure specific layout to the input arguments. Consequently, the trade-off of optimizing for minimal conversion burden on the end user is more processing and higher memory usage. However, this extra cost at the post processing is vanishingly small compared to the original cost of running numerical models.

With the aforementioned principle in mind, all that is expected from the user is to have access to, at least, the following arrays:

- the Cartesian x, y, and z coordinates (will refer to it as `input_xs`, `input_ys`, and `input_zs`),
- as well as the corresponding field values (will refer to it as `input_field`).

Furthermore, though all input arrays are assumed to be `numpy` (Harris et al., 2020) arrays, they can have any shape (flat or multidimensional).

The fist step is to generate a grid:

```
from platerecipy.grid import SphericalGrid

# generating a consistent grid for interpolation
grid = SphericalGrid(input_xs, input_ys, input_zs)
```

The `SphericalGrid` object is itself a derived type of a generic class `Grid`, but it informs the code whether additional considerations (e.g., applying the wraparound boundary condition or spherical distances) are required. Then, our `SphericalGrid` object is to be used to interpolate `input_field` to the input points to a spherical grid. The segmentation is performed on the linearly-interpolated uniform spherical grid. The interpolation process is performed using the method `interpolate_field`:

```
# interpolating an input field
field = grid.interpolate_field(input_field)
```

It should be noted that for input fields such as viscosity or strain-rate (which vary by orders of magnitude), the optional argument `take_log` should be set to `True` in order to the linear interpolation is performed in the log space (and then converted back).

With the `grid` object and the interpolated field `field`, we have the necessary arguments to perform plate detection. To initiate the process, a `PlateModel` object is to be constructed as follows:

```
from platerecipy.model import PlateModel

# initializing a plate model
m = PlateModel(grid)
```

The newly-created object `m` carries (a pointer) to the `grid` object which includes information about the original input as well as the interpolated grid structure. Though it would have been possible to keep interpolation under the hood (given `PlateModel` carries a `Grid` attribute), the user may want to further manipulate the interpolated field before passing it for plate detection. That is why the two steps are kept separate. The importance of the class `PlateModel` is in the fundamental idea that what defines plate interior/boundary must be made explicit and consistent across subsequent snapshots of the same model. The `PlateModel` class has the ability to stack several fields with

different corresponding weights and normalize the stacked field to span $[0, 1]$. Stacking fields are done through the `PlateModel.stack_field()` method as follows:

```
# stacking the interpolated field
m.stack_field(field)
```

Once again, for fields that vary by orders of magnitude, `PlateModel.stack_field()` accepts an optional argument `take_log` which if set to `True` it takes the logarithm to flatten the distribution. Most importantly, the stacked fields are assumed to be markers of deformation such that 0 and 1 correspond to minimal and maximal deformation. For a field that is opposite of this (i.e., its high corresponds to low deformation and vice versa), there is an additional argument `invert` which ensures the stacked field stays consistent.

Once all the fields (if multiple) are stacked, the `PlateModel.find_plates()` method performs segmentation:

```
# finding plates on the stacked field
plate_IDs = m.find_plates(
    boundary_quantile  = 0.9,          # threshold for the boundaries
    # ...
)
```

The `PlateModel.find_plates()` method both returns the segmentation result, and also it save them in the model object as `m.plate_IDs` (containing segmentation IDs) and `m.ID_probs` (containing the associated probabilities).

5 Detailed API documentation

Though this guide is maintained such that with every modification the corresponding section is updated, in case of any discrepancies, function docstrings are to be considered the main source providing updated description. In order to access docstrings, one can easily invoke `help()` command or use `?` on `jupyter` notebook.

5.1 Overall structure

The source code is effectively divided to two portions:

- `src/platerecipy`: where the `Python` modules reside, and
- `src/clib`: where various backend utilities are implemented in `C` for better performance.

5.2 `grid` module

The `grid` module contains the necessary class and function definitions for grid structure. It contains the generic `Grid` class inherited by `SphericalGrid` class which has the additional attributes relevant for a spherical surface. As a general rule, θ and ϕ correspond to polar (colatitude) and azimuthal (longitude) directions.

```
class SphericalGrid(Grid):
    """
    Uniform spherical grid.
    """

    def __init__(
        self,
        original_xs  : np.ndarray,
        original_ys  : np.ndarray,
        original_zs  : np.ndarray,
        theta_res    = None,
        phi_res      = None,
    )
        """
        Constructs a uniform spherical grid structure.

        If `theta_res` and `phi_res` are provided, the resolution will be
        determined by those values. Alternatively, the resolution close enough
        to what provided by the Cartesian arguments.

        Warning
        -------
        The Cartesian arguments will be the basis upon which the class method
        `interpolate_field()` operates. In consistent values will impact the
        interpolation process.

        Parameters
        ----------
        original_xs : np.ndarray,
            array of original input Cartesian x coordinates.

        original_ys : np.ndarray,
```

```
            array of original input Cartesian y coordinates.

        original_zs : np.ndarray,
            array of original input Cartesian z coordinates.

        theta_res : int, optional
            polar (colatitude) resolution.

        phi_res : int, optional
            azimuthal (longitude) resolution.
        """

    def interpolate_field(
        self,
        field       : np.ndarray,
        take_log    = False
    ) -> np.ndarray:
        """
        Interpolates the input field linearly for convex hull interior points and
        closest points to the exterior.

        Parameters
        ----------
        field : np.ndarray,
            array corresponding to the original Cartesian coordinate inputs.

        take_log : bool, False
            whether to perform the interpolation on the log space (suitable for
            fields that vary by orders of magnitude), default = False.

        Returns
        -------
        np.ndarray
        """
```

5.3 `model` module

5.4 `io` module

5.5 `transform` module

6 Examples

```python
from platerecipy.model import PlateModel
from platerecipy.grid import SphericalGrid

# generating a consistent grid for interpolation
grid = SphericalGrid(input_xs, input_ys, input_zs)

# interpolating an input field
field = grid.interpolate_field(input_field)

# initializing a plate model
m = PlateModel(grid)

# stacking the interpolated field
m.stack_field(field, take_log=True)

# finding plates on the stacked field
m.find_plates(
    boundary_quantile  = 0.9,          # threshold for the boundaries
    # ...
)

# outputting as a ParaView readable .vtp file
from platerecipy import io
io.save_as_vtp(m)
```

References

Grady, L. (2006). Random walks for image segmentation. *IEEE transactions on pattern analysis and machine intelligence*, *28*(11), 1768–1783.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Sullivan, C. B., & Kaszynski, A. (2019). PyVista: 3d plotting and mesh analysis through a streamlined interface for the visualization toolkit (VTK). *Journal of Open Source Software*, *4*(37), 1450. https://doi.org/10.21105/joss.01450

van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., Yu, T., & the scikit-image contributors. (2014). Scikit-image: Image processing in Python. *PeerJ*, *2*, e453. https://doi.org/10.7717/peerj.453

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

A About the backend